



Bootcamp CDIA – Ciência de Dados e Inteligência Artificial

Residente: Diego Oliveira de Araújo

Data: 31/05/2026

Tema: Desenvolvimento de modelos de Inteligência Artificial e Visão Computacional para aplicações industriais, contemplando dois estudos de caso: Sistema de detecção e contagem automática de parafusos e Sistema de detecção e localização de trincas e fissuras.

1. INTRODUÇÃO

A Inteligência Artificial (IA) e a Visão Computacional têm se destacado como tecnologias fundamentais para a modernização da indústria, permitindo a automatização de tarefas de inspeção, monitoramento e controle de qualidade por meio da análise de imagens. Segundo Andrew Ng, uma das principais referências da área, “a IA é a nova eletricidade”, destacando seu potencial de transformação em diversos setores. Nesse contexto, a aplicação dessas tecnologias possibilita aumentar a eficiência operacional, reduzir custos e aprimorar a tomada de decisões em ambientes industriais.

2. DESAFIOS

DESAFIO 1 (Sistema de detecção e contagem automática de parafusos)

O conjunto inicial disponibilizado pela empresa continha apenas cinco imagens, quantidade insuficiente para o treinamento de um modelo de detecção de objetos com capacidade de generalização adequada. Nos primeiros experimentos, utilizando exclusivamente essas imagens e técnicas de data augmentation, o modelo apresentou resultados inconsistentes.

Foram observados casos em que diferentes partes de um mesmo parafuso eram identificadas como objetos distintos, fazendo com que a cabeça e o corpo do parafuso fossem contabilizados como dois ou até três itens diferentes. Além disso, ocorreram falsos positivos, nos quais o modelo detectava parafusos em regiões da imagem onde não havia qualquer objeto de interesse. Esses comportamentos evidenciaram que o conjunto de dados inicial não possuía diversidade e volume suficientes para que o modelo aprendesse de forma robusta as características visuais dos parafusos.

Para superar essa limitação, foi adotada uma estratégia de enriquecimento do conjunto de dados por meio da incorporação de imagens públicas de parafusos disponíveis na plataforma Roboflow Universe ([Counting Screws Computer Vision Dataset](#)), complementadas pelas imagens fornecidas pela empresa e por técnicas de data augmentation ([data fusion](#)). Essa abordagem possibilitou ampliar significativamente a variabilidade do dataset, contemplando diferentes formatos, tamanhos, posições, orientações, condições de

iluminação e contextos de captura. O Roboflow também foi utilizado como ferramenta de apoio para gerenciamento das anotações, organização do conjunto de dados e aplicação de transformações automáticas, acelerando o processo de preparação dos dados para treinamento.

Como arquitetura de visão computacional, foi escolhida o YOLO (You Only Look Once), uma das abordagens mais consolidadas para detecção de objetos em tempo real. A escolha foi motivada pela combinação entre elevada precisão, velocidade de inferência e baixo custo computacional, portanto, o treinamento foi realizado utilizando a arquitetura YOLOv8n, versão "nano" da família YOLOv8. A escolha desse modelo foi motivada principalmente pelo equilíbrio entre desempenho e eficiência computacional, já que um dos requisitos do desafio é a possibilidade de utilização da solução nos smartphones já empregados pela equipe do setor. Embora existam versões maiores, como YOLOv8s, YOLOv8m e YOLOv8l, essas arquiteturas demandam mais recursos computacionais e maior capacidade de processamento durante a inferência, o que pode dificultar sua utilização em dispositivos móveis.

Dessa forma, a solução proposta não apenas atende aos requisitos de acurácia, mas também apresenta potencial de implantação em dispositivos móveis com recursos computacionais limitados.

Após o enriquecimento do dataset e o treinamento do modelo, observou-se uma evolução significativa em relação aos experimentos iniciais. O modelo alcançou precisão (Precision) de 96,51%, indicando que a grande maioria das detecções realizadas corresponde efetivamente a parafusos. O Recall obtido foi de 95,56%, demonstrando elevada capacidade de identificar os objetos presentes nas imagens e reduzindo significativamente a ocorrência de itens não contabilizados durante o processo de contagem.

Os resultados de $mAP@50$ e $mAP@50-95$ foram de 96,38% e 83,99%, respectivamente. O elevado valor de $mAP@50$ demonstra excelente capacidade de detecção e localização dos parafusos, enquanto o resultado de $mAP@50-95$ evidencia que o modelo mantém desempenho consistente mesmo sob critérios de avaliação mais rigorosos, que exigem maior precisão no posicionamento das bounding boxes. O valor de fitness de 83,99% reforça a qualidade geral do modelo treinado e sua capacidade de generalização.

Outro aspecto importante é que a avaliação foi realizada sobre um conjunto contendo 723 parafusos anotados, distribuídos em múltiplas imagens de validação. Isso confere maior robustez estatística às métricas obtidas, tornando os resultados mais representativos do comportamento esperado do modelo em cenários reais de utilização. Diferentemente dos testes iniciais realizados apenas com as cinco imagens fornecidas pela empresa, a avaliação final foi conduzida em um conjunto significativamente mais amplo e diversificado, permitindo uma análise mais confiável da capacidade de generalização da solução.

Os resultados demonstram que a estratégia de enriquecimento do dataset foi determinante para reduzir problemas de dupla contagem e falsos positivos observados nas primeiras versões do modelo. A utilização combinada de dados públicos, imagens do cenário real da empresa e técnicas de augmentation permitiu construir uma base de treinamento mais representativa, resultando em um sistema capaz de detectar e contar parafusos com elevado nível de precisão.

Dessa forma, a solução desenvolvida configura uma prova de conceito sólida para aplicação no processo de coleta/solução da empresa, contribuindo para a redução de erros humanos, aumento da confiabilidade na conferência dos itens coletados e maior eficiência operacional no processo de separação e envio de produtos aos clientes.

Durante os experimentos iniciais, também foram realizados testes com modelos maiores da família YOLO. No entanto, utilizando a GPU NVIDIA T4 disponibilizada gratuitamente pelo Google Colab, verificou-se que esses modelos demandavam tempos de treinamento significativamente maiores e, em alguns casos, ultrapassavam os limites de recursos e tempo da plataforma. Considerando as restrições computacionais disponíveis e o

objetivo de desenvolver uma solução passível de execução em smartphones, a YOLOv8n mostrou-se a melhor relação entre custo computacional e desempenho.

O treinamento foi configurado para 100 épocas (epochs), permitindo que o modelo realizasse múltiplas passagens pelo conjunto de treinamento para aprender os padrões visuais dos parafusos. Esse valor foi definido como um equilíbrio entre tempo de treinamento e desempenho do modelo. Quantidades muito reduzidas de épocas poderiam resultar em subtreinamento (underfitting), enquanto valores excessivamente elevados aumentariam o risco de sobreajuste (overfitting), especialmente considerando que parte do conjunto de dados foi ampliada por meio de técnicas de data augmentation. Os experimentos realizados demonstraram que 100 épocas eram suficientes para alcançar a convergência do modelo e obter métricas satisfatórias de desempenho.

O parâmetro de resolução das imagens (imgsz) foi definido como 800 pixels. Essa configuração foi escolhida para preservar detalhes importantes dos parafusos, permitindo que o modelo identificasse corretamente suas características visuais mesmo em situações de proximidade entre objetos ou variações de escala. Como os parafusos representam objetos relativamente pequenos dentro da imagem, uma resolução superior ao padrão de 640 pixels contribui para melhorar a qualidade das detecções. Ao mesmo tempo, a resolução de 800 pixels manteve o treinamento viável dentro das limitações computacionais da GPU T4 utilizada.

O tamanho do lote (batch size) foi definido como 4 imagens por iteração. Essa escolha foi diretamente influenciada pela memória disponível na GPU durante o treinamento. Valores maiores resultavam em elevado consumo de memória, aumentando o risco de interrupções ou falhas na execução. Por outro lado, um batch size muito pequeno poderia tornar o processo de treinamento mais instável. O valor adotado permitiu utilizar a resolução de 800 pixels de forma estável, aproveitando adequadamente os recursos computacionais disponíveis sem comprometer a qualidade do treinamento.

As decisões relacionadas à arquitetura do modelo, número de épocas, resolução das imagens e tamanho do lote foram tomadas com base em testes experimentais e nas restrições do ambiente de desenvolvimento. O objetivo foi obter um modelo com elevada capacidade de detecção e contagem de parafusos, mantendo a viabilidade de treinamento em infraestrutura acessível e a possibilidade de implantação futura em smartphones utilizados pela equipe de coleta/seleção da empresa.

DESAFIO 2 (Sistema de detecção e localização de trincas e fissuras)

Para o desafio de identificação de fissuras, foi adotada uma abordagem baseada em segmentação de instâncias, diferentemente do desafio de contagem de parafusos, que utilizava apenas detecção de objetos. A escolha pela segmentação foi motivada pelas características próprias das fissuras, que apresentam formatos irregulares, geometrias complexas e espessuras variáveis ao longo de sua extensão. Em aplicações de inspeção industrial e estrutural, não é suficiente apenas identificar a presença de uma fissura; é desejável também determinar com maior precisão a região ocupada pelo defeito, possibilitando análises futuras relacionadas à extensão, área afetada e evolução do dano.

Na detecção tradicional, o modelo retorna apenas uma *bounding box* envolvendo o objeto identificado. Nesse cenário, o sistema informa que existe uma fissura em determinada região da imagem, porém não define exatamente quais pixels pertencem ao defeito. Já na segmentação, além da localização, o modelo produz uma máscara (*mask*) que delimita precisamente a área correspondente à fissura. Em outras palavras, enquanto a detecção responde "onde está a fissura", a segmentação busca responder "qual é exatamente o formato da fissura".

Essa diferença é particularmente importante para fissuras, pois se tratam de objetos extremamente finos, alongados e com contornos irregulares. Pequenas variações na anotação das máscaras podem impactar significativamente as métricas de avaliação. Por esse motivo, as métricas de segmentação tendem a ser naturalmente mais rigorosas do que as métricas de detecção. É relativamente simples para o modelo localizar a região onde existe uma fissura, porém é muito mais difícil reproduzir com precisão todos os seus contornos, ramificações e variações de espessura.

O conjunto de dados utilizado no treinamento foi composto por 1323 imagens para treinamento e 311 imagens para validação, totalizando 427 fissuras anotadas distribuídas entre as imagens avaliadas. Esse volume de dados mostrou-se suficiente para que o modelo aprendesse a identificar os padrões visuais associados às fissuras, permitindo a realização de experimentos comparativos com diferentes configurações de treinamento.

Como arquitetura de visão computacional, foi utilizada a família YOLOv8 com suporte à segmentação (*YOLOv8-seg*). Essa arquitetura foi escolhida por combinar elevada velocidade de inferência, facilidade de implantação e capacidade de executar simultaneamente tarefas de detecção e segmentação.

Durante os experimentos, foram avaliadas três configurações distintas de treinamento, buscando encontrar o melhor equilíbrio entre desempenho e custo computacional.

A primeira configuração utilizou o modelo YOLOv8s-seg com resolução de entrada de 768 pixels e apenas 20 épocas de treinamento. Nessa configuração, foram obtidos valores de precisão de 77,98% e recall de 66,28% para detecção, além de mAP@50 de 72,95% e mAP@50-95 de 52,04%. Para segmentação, os resultados foram de 67,62% de precisão, 56,21% de recall, mAP@50 de 54,62% e mAP@50-95 de 19,92%. A análise das curvas de treinamento demonstrou que o modelo ainda apresentava crescimento contínuo das métricas ao final da vigésima época, indicando que o processo de aprendizagem não havia convergido. Dessa forma, concluiu-se que a quantidade de épocas utilizada foi insuficiente para explorar plenamente a capacidade do modelo.

Na segunda configuração, manteve-se a arquitetura YOLOv8s-seg e a resolução de 768 pixels, porém com aumento do número de épocas. Os resultados evoluíram significativamente, alcançando precisão de 84,51% e recall de 71,19% para detecção, mAP@50 de 78,33% e mAP@50-95 de 60,80%. Para segmentação, foram obtidos 76,21% de precisão, 63,76% de recall, mAP@50 de 64,21% e mAP@50-95 de 24,61%. A melhora observada confirmou que a principal limitação da configuração anterior estava associada ao subtreinamento (*underfitting*), uma vez que o modelo ainda não havia recebido épocas suficientes para consolidar os padrões aprendidos.

A terceira configuração utilizou resolução de 640 pixels, batch size de 8 imagens por iteração e 100 épocas de treinamento. Essa configuração apresentou os melhores resultados globais entre todos os experimentos realizados. Foram obtidos 86,81% de precisão e 73,07% de recall para detecção, com mAP@50 de 80,23% e mAP@50-95 de 64,05%. Na tarefa de segmentação, os resultados alcançaram 76,71% de precisão, 65,55% de recall, mAP@50 de 65,82% e mAP@50-95 de 26,60%. O valor de *fitness* foi de 90,64%, representando o melhor desempenho geral observado durante os experimentos.

Um resultado particularmente interessante foi a constatação de que a resolução de 768 pixels não proporcionou ganhos suficientes para compensar o aumento do custo computacional quando comparada à resolução de 640 pixels. Embora seja comum associar resoluções maiores a melhor desempenho em tarefas de segmentação, os experimentos

demonstraram que, para o conjunto de dados utilizado, o aumento do número de épocas exerceu influência mais significativa sobre os resultados do que o aumento da resolução das imagens. Além disso, resoluções mais elevadas demandam maior consumo de memória da GPU e tempos de treinamento substancialmente maiores.

Durante os testes também foi avaliada a possibilidade de utilizar modelos mais robustos da família YOLO, como versões superiores da arquitetura de segmentação.

Entretanto, ao utilizar a GPU NVIDIA T4 disponibilizada gratuitamente pelo Google Colab, verificou-se que configurações mais pesadas, associadas a resoluções elevadas, resultavam em tempos de treinamento superiores a cinco horas e, em alguns casos, excediam os limites de utilização da plataforma antes da conclusão do treinamento. Dessa forma, tais configurações foram descartadas por inviabilidade computacional.

Outro aspecto relevante diz respeito à interpretação das métricas de segmentação. Os resultados demonstraram que o modelo alcançou desempenho satisfatório para localizar fissuras, apresentando valores elevados de $mAP@50$. Entretanto, os valores de $mAP@50-95$ para segmentação permaneceram significativamente inferiores aos observados para detecção. Esse comportamento é esperado em aplicações envolvendo fissuras, pois a métrica $mAP@50-95$ exige elevada precisão na correspondência entre os contornos previstos pelo modelo e os contornos reais presentes nas máscaras anotadas. Em termos práticos, isso significa que o modelo consegue identificar corretamente a existência e a localização das fissuras, porém ainda apresenta dificuldades em reproduzir exatamente todos os detalhes de seus contornos.

Essa característica está diretamente relacionada à natureza do problema. Fissuras são objetos estreitos, descontínuos e frequentemente apresentam espessura de apenas alguns pixels. Pequenas diferenças entre a máscara prevista e a máscara real produzem reduções significativas nas métricas mais rigorosas. Assim, embora o modelo apresente elevada capacidade de identificação das fissuras, ainda existem oportunidades de melhoria relacionadas ao refinamento das máscaras e à representação mais precisa dos contornos.

Considerando simultaneamente desempenho, custo computacional e viabilidade de reprodução dos experimentos, a configuração composta por 100 épocas, resolução de 640 pixels e batch size de 8 imagens mostrou-se a mais adequada para o problema estudado.

Além de apresentar os melhores resultados quantitativos, essa configuração concluiu o treinamento dentro das limitações da infraestrutura disponível e demonstrou capacidade de generalização em imagens externas ao conjunto de treinamento, indicando potencial para utilização futura como prova de conceito em aplicações reais de inspeção e monitoramento de fissuras.

3. DESENVOLVIMENTO

APP MOBILE

O aplicativo mobile foi desenvolvido utilizando o framework Kivy, permitindo a execução da solução em dispositivos móveis Android. O objetivo do aplicativo é fornecer uma interface simples para que o usuário possa capturar ou selecionar imagens diretamente do dispositivo e enviá-las para processamento pela API de inteligência artificial.

As principais funcionalidades implementadas foram:

- Interface gráfica para interação do usuário;
- Seleção de imagens armazenadas no dispositivo;
- Comunicação com a API através de requisições HTTP;
- Envio de imagens para análise utilizando o método POST;
- Recebimento e exibição dos resultados processados pela inteligência artificial;
- Apresentação da imagem anotada contendo as detecções realizadas pelo modelo.
- Seleção entre os módulos de detecção de parafusos e identificação de fissuras.

O aplicativo atua exclusivamente como cliente da API, não executando processamento de inteligência artificial localmente.



Img 1 – Aplicativo Mobile

APP WEB

A aplicação web foi desenvolvida utilizando Streamlit, framework voltado para criação rápida de interfaces interativas em Python.

O sistema web permite que usuários realizem análises diretamente pelo navegador, sem necessidade de instalação de software adicional.

As funcionalidades desenvolvidas incluem:

- Upload de imagens em formato JPG e PNG;
- Comunicação com a API de inferência através de requisições REST;
- Exibição dos resultados retornados pelo servidor;
- Visualização da imagem processada contendo as detecções e segmentações geradas pelos modelos de visão computacional;
- Seleção entre os módulos de detecção de parafusos e identificação de fissuras.

A aplicação foi preparada para implantação na plataforma Streamlit Cloud, permitindo acesso remoto via navegador.



Img 2 – Aplicativo Web

FAST API

O backend da solução foi desenvolvido utilizando o framework FastAPI, responsável por disponibilizar serviços REST para comunicação entre os clientes e os modelos de inteligência artificial.

O FastAPI atua como camada intermediária entre as aplicações cliente e os modelos de visão computacional.

Foram implementados os seguintes endpoints:

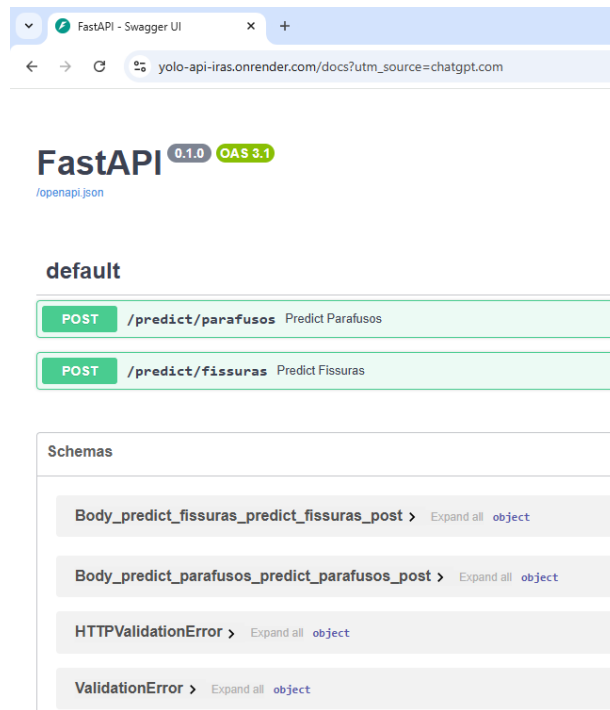
/predict/parafusos

/predict/fissuras

O uso do FastAPI foi escolhido devido ao seu alto desempenho, facilidade de integração com modelos de machine learning e suporte nativo à documentação automática de APIs.

As responsabilidades da API incluem:

- Recebimento das imagens enviadas pelos clientes;
- Conversão dos arquivos recebidos para formato processável;
- Carregamento dos modelos YOLO treinados;
- Execução da inferência;
- Geração da imagem anotada com as detecções;
- Conversão da imagem para Base64;
- Retorno dos resultados em formato JSON.



Img 3 – API

INTELIGÊNCIA ARTIFICIAL/VISÃO COMPUTACIONAL

O desenvolvimento dos modelos de Inteligência Artificial e Visão Computacional constituiu a etapa central do projeto, sendo responsável pela automatização da análise de imagens industriais por meio de técnicas de Deep Learning.

A solução foi desenvolvida utilizando a arquitetura **YOLOv8 (You Only Look Once)**, amplamente empregada em aplicações de detecção e segmentação de objetos em tempo real devido ao seu equilíbrio entre precisão e desempenho computacional.

Para a preparação dos dados, foi utilizada a plataforma **Roboflow**, responsável pela anotação das imagens, organização do dataset, separação dos conjuntos de treinamento e validação, aplicação de técnicas de aumento de dados (Data Augmentation) e exportação para o formato compatível com a arquitetura YOLOv8. Essas etapas contribuíram para aumentar a variabilidade das amostras e melhorar a capacidade de generalização dos modelos.

Foram desenvolvidos dois modelos independentes:

Modelo de Detecção de Parafusos

O primeiro modelo foi treinado para identificar e contabilizar parafusos presentes em imagens industriais. Para isso, foi realizado o processo de anotação dos objetos, criação do conjunto de treinamento e treinamento supervisionado utilizando imagens rotuladas.

O modelo resultante é capaz de:

- Detectar automaticamente os parafusos presentes na imagem;
- Delimitar cada objeto identificado através de bounding boxes;
- Fornecer o nível de confiança de cada detecção;
- Realizar a contagem automática dos elementos encontrados.

Modelo de Segmentação de Fissuras

O segundo modelo foi desenvolvido para identificação de fissuras em superfícies industriais.

Diferentemente da detecção convencional, foi utilizada a abordagem de segmentação de instâncias, permitindo que o modelo identifique não apenas a localização da fissura, mas também seu contorno exato na imagem.

As principais funcionalidades incluem:

- Identificação automática de fissuras;
- Delimitação precisa das regiões afetadas;
- Geração de máscaras de segmentação;
- Apoio à inspeção visual automatizada.

CLOUD

A infraestrutura em nuvem foi projetada para permitir a execução remota da aplicação, possibilitando que múltiplos usuários realizem inferências sem necessidade de processamento local.

Foram utilizadas as seguintes plataformas:

Streamlit Cloud

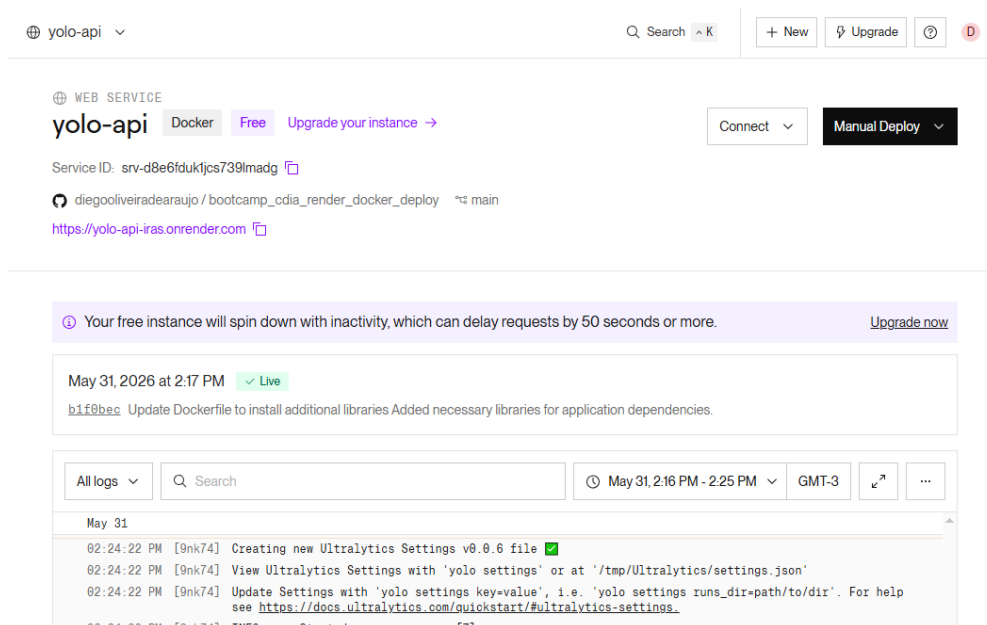
Responsável pela hospedagem da aplicação web, permitindo acesso via navegador.

Render Cloud

Responsável pela hospedagem do backend e dos modelos de inteligência artificial.

A arquitetura em nuvem oferece:

- Disponibilidade remota da solução;
- Centralização dos modelos de IA;
- Facilidade de manutenção e atualização;
- Escalabilidade para futuras expansões do sistema;
- Acesso simultâneo por múltiplos dispositivos.
- Toda comunicação entre cliente e servidor é realizada por meio do protocolo HTTPS.



Img 4 – Render Cloud/Docker

MICROSSERVIÇO

A solução foi estruturada seguindo princípios de arquitetura baseada em microsserviços, promovendo desacoplamento entre as diferentes camadas do sistema.

O microsserviço principal é composto pelo servidor FastAPI encapsulado em um container Docker, responsável pela execução dos modelos de visão computacional.

As principais características da arquitetura incluem:

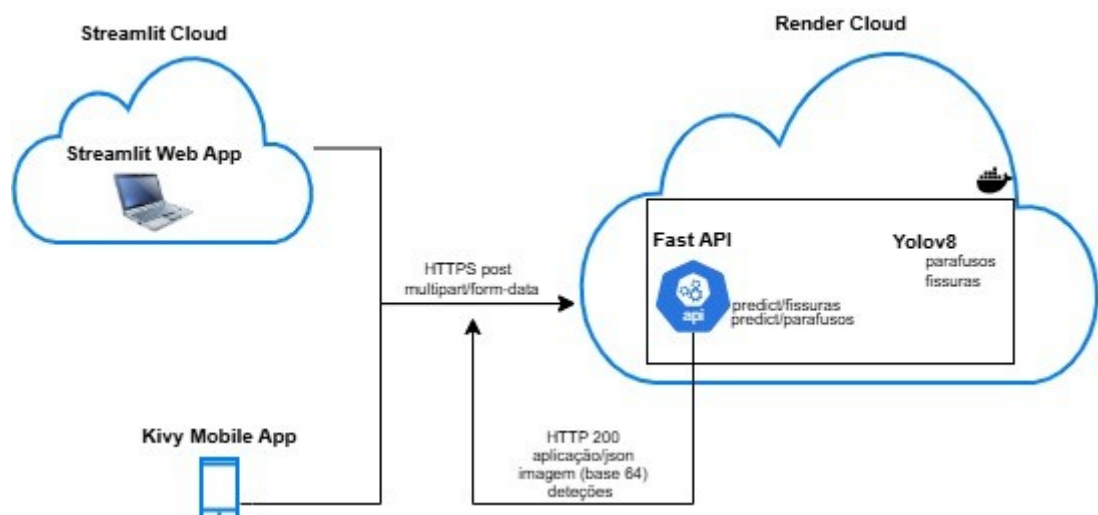
- Separação entre frontend e backend;
- Independência entre aplicação web e aplicação mobile;
- Centralização da lógica de inteligência artificial;
- Exposição dos serviços por meio de endpoints REST;
- Facilidade de manutenção e atualização dos modelos;
- Possibilidade de inclusão de novos modelos sem necessidade de alteração nos clientes.

A utilização de Docker garante portabilidade e padronização do ambiente de execução, permitindo que o serviço seja implantado em diferentes provedores de nuvem com mínima necessidade de adaptação.

O fluxo operacional do microsserviço ocorre da seguinte forma:

1. O cliente envia uma imagem para a API.
2. O FastAPI recebe e valida o arquivo.
3. O modelo YOLO correspondente é carregado.
4. A inferência é executada.
5. Os resultados são processados.
6. A imagem anotada é gerada.
7. O resultado é retornado em formato JSON para o cliente.

Essa arquitetura proporciona modularidade, escalabilidade e facilidade de evolução da plataforma de inteligência artificial.



Img 5 – Arquitetura do Projeto

4. RECURSOS UTILIZADOS

Windows 10
Linux/Debian 12
Google Colab
Python 3.11.4
Git 2.41.0
GitHub
Streamlit Cloud
Render Cloud
Docker

5. REPOSITÓRIOS

Aplicação Web (Streamlit Cloud):
https://github.com/diegooliveiradearaujo/bootcamp_cdia_streamlit_cloud.git

Containerização API e IA (Render Cloud):
https://github.com/diegooliveiradearaujo/bootcamp_cdia_render_docker_deploy.git

Desenvolvimento do projeto (todos os recursos):
https://github.com/diegooliveiradearaujo/bootcamp_cdia_ia_industrial.git

6. CONCLUSÃO

O desenvolvimento dos dois estudos de caso demonstrou o potencial da Inteligência Artificial e da Visão Computacional para automatizar processos industriais tradicionalmente dependentes de inspeção manual. Os resultados obtidos evidenciaram que a arquitetura YOLOv8 é capaz de atender diferentes demandas, desde a detecção e contagem automática de parafusos até a identificação e segmentação de fissuras, apresentando desempenho satisfatório mesmo em ambientes com limitações computacionais.

Além do desenvolvimento dos modelos de IA, o projeto contemplou a construção de uma solução completa, incluindo aplicações web e mobile, API de inferência, infraestrutura em nuvem e arquitetura baseada em microsserviços. Essa abordagem permitiu transformar os modelos treinados em aplicações acessíveis e prontas para utilização em cenários reais.

Os experimentos realizados também evidenciaram a importância da qualidade e diversidade dos dados, bem como da definição adequada dos parâmetros de treinamento, para obtenção de modelos robustos e com capacidade de generalização. Dessa forma, as soluções desenvolvidas constituem provas de conceito viáveis para aplicações industriais, contribuindo para a redução de erros operacionais, aumento da produtividade, melhoria dos processos de inspeção e apoio à tomada de decisão baseada em dados.